Coding Convention & Git Convention

Project: End-to-End Email Spam Classification

Project Type: AI / Machine Learning / MLOps Demo Project
Main Stack: Python, Pandas, Scikit-learn, MLflow, FastAPI, Streamlit, Docker
Main Model: TF-IDF + Logistic Regression
Problem Type: Binary Text Classification
Labels: `0 = ham / not_spam`, `1 = spam`

---

1. Purpose

This document defines coding standards, repository structure, Git workflow, branch naming, commit convention, pull request rules, and review rules for the **Email Spam Classification** project.

The goals are:

- Keep code readable and consistent.
- Make collaboration easier across 5 roles.
- Reduce merge conflicts.
- Make code easier to review and test.
- Avoid leaking secrets or committing large generated files.
- Support a clean end-to-end ML workflow from data to model serving.

---

2. Team Roles and Ownership

Role	Main Scope
Tech Lead	Architecture, code review, integration, deployment
QA / Reviewer	Test plan, acceptance criteria, output review, final QA report
AI Engineer — Data	Dataset, EDA, Bronze/Silver/Gold data pipeline
AI Engineer — Training Pipeline	Training pipeline, MLflow, tuning, model artifact
AI Engineer — Model Serving	Inference, FastAPI, Streamlit, prediction logging

Ownership rule:

```
Data role owns: data pipeline outputs
Pipeline role owns: training and model artifact
Model Serving role owns: prediction API and UI
Tech Lead owns: integration and deployment
QA owns: validation and quality control
```

3. Project Structure Convention

Recommended structure:

```
email-spam-classification/
│
├── data/
│   ├── raw/
│   ├── bronze/
│   ├── silver/
│   ├── gold/
│   └── test_samples/
│
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_baseline_logistic_regression.ipynb
│   ├── 03_baseline_naive_bayes.ipynb
│   ├── 04_baseline_knn.ipynb
│   └── 05_model_comparison.ipynb
│
├── src/
│   ├── __init__.py
│   ├── bronze_ingest.py
│   ├── silver_transform.py
│   ├── gold_build.py
│   ├── text_preprocessing.py
│   ├── train.py
│   ├── evaluate.py
│   ├── tune.py
│   └── predict.py
│
├── app/
│   ├── api.py
│   └── streamlit_app.py
│
├── models/
│   ├── best_spam_classifier.pkl
│   └── model_metadata.json
│
├── reports/
│   ├── data_report.md
│   ├── model_report.md
│   ├── api_ui_report.md
│   └── final_test_report.md
│
├── logs/
│   └── predictions.csv
│
├── tests/
│   ├── test_data_pipeline.py
│   ├── test_predict.py
```

```
│   └── test_api.py
│
├── docs/
│   ├── fsd.md
│   ├── tsd.md
│   ├── architecture.md
│   ├── coding_and_git_convention.md
│   ├── test_plan.md
│   └── deployment_guide.md
│
├── README.md
├── requirements.txt
├── .gitignore
├── .env.example
├── Dockerfile
└── docker-compose.yml
```

---

4. Python Coding Convention

4.1 Python Version

Use:

```
Python 3.10+
```

Recommended:

```
Python 3.10 or 3.11
```

4.2 Code Style

Follow PEP 8 style.

Rules:

- Use 4 spaces for indentation.
- Use English for variable names, function names, class names, comments, and docstrings.
- Keep functions small and focused.
- Avoid deeply nested logic.
- Avoid unused imports.
- Avoid hard-coded absolute paths.
- Avoid committing secrets, API keys, passwords, or tokens.

Recommended tools:

```
pip install black isort flake8 pytest
```

Before pushing code, run:

```
black .
isort .
flake8 .
pytest
```

---

5. Naming Convention

5.1 Files and Folders

Use `snake_case`.

Good:

```
bronze_ingest.py
silver_transform.py
gold_build.py
text_preprocessing.py
train.py
evaluate.py
streamlit_app.py
```

Bad:

```
BronzeIngest.py
silverTransform.py
gold-build.py
TrainModel.py
```

5.2 Variables

Use `snake_case`.

Good:

```
raw_data_path = "data/raw/spam.csv"
model_path = "models/best_spam_classifier.pkl"
```

```
spam_probability = 0.94
```

Bad:

```
rawDataPath = "data/raw/spam.csv"
ModelPath = "models/best_spam_classifier.pkl"
spamProbability = 0.94
```

5.3 Functions

Use `snake_case` and start with a verb when possible.

Good:

```
def load_dataset(file_path: str):
    pass


def clean_text(text: str) -> str:
    pass


def train_model():
    pass


def evaluate_model():
    pass


def predict_spam(text: str):
    pass
```

Bad:

```
def data():
    pass


def model():
    pass


def run():
    pass
```

5.4 Classes

Use `PascalCase`.

Good:

```
class EmailInput:
    pass


class PredictionResponse:
    pass
```

FastAPI/Pydantic example:

```
from pydantic import BaseModel


class EmailInput(BaseModel):
    subject: str = ""
    body: str = ""
```

5.5 Constants

Use `UPPER_SNAKE_CASE`.

Good:

```
DATA_PATH = "data/raw/spam.csv"
MODEL_PATH = "models/best_spam_classifier.pkl"
RANDOM_STATE = 42
TEST_SIZE = 0.2
```

---

6. Import Convention

Use this import order:

```
# Standard library
import os
import re
from pathlib import Path

# Third-party libraries
import joblib
```

```python
import pandas as pd
from sklearn.pipeline import Pipeline

# Local modules
from src.text_preprocessing import clean_text
from src.predict import predict_spam
```

Do not use wildcard imports:

```python
# Bad
from sklearn.metrics import *
```

Use explicit imports:

```python
# Good
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score
```

---

7. Function and Module Design

7.1 One Function, One Responsibility

Good:

```python
def clean_text(text: str) -> str:
    text = text.lower()
    text = re.sub(r"http\S+|www\S+", " URL ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

Avoid writing a single function that does everything:

```python
# Bad
def run_all():
    # load data
    # clean data
    # train model
    # evaluate model
    # save model
    # start API
    pass
```

7.2 Type Hints

Use type hints for important functions.

Example:

```
from pathlib import Path
import pandas as pd


def load_dataset(file_path: Path) -> pd.DataFrame:
    return pd.read_csv(file_path)
```

Prediction example:

```
def predict_spam(text: str) -> dict:
    prediction = model.predict([text])[0]
    probability = model.predict_proba([text])[0][1]

    return {
        "label": "spam" if prediction == 1 else "not_spam",
        "spam_probability": float(probability),
    }
```

---

8. Comment and Docstring Convention

8.1 Comments

Comments should explain **why**, not repeat what the code already says.

Good:

```
# Use stratify to preserve spam/not_spam ratio in both train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y,
)
```

Bad:

```
# Read CSV
```

```
train_df = pd.read_csv("data/train.csv")
```

8.2 Docstrings

Use docstrings for public functions or important pipeline functions.

Example:

```
def clean_text(text: str) -> str:
    """
    Clean email text before TF-IDF feature extraction.

    Args:
        text: Raw email text.

    Returns:
        Cleaned text string.
    """
    ...
```

---

9. Data Convention

9.1 Dataset Layers

This project uses local Medallion Architecture:

Layer	Path	Meaning
Raw	`data/raw/`	Original downloaded dataset
Bronze	`data/bronze/`	Raw ingested data
Silver	`data/silver/`	Cleaned and standardized data
Gold	`data/gold/`	Model-ready train/test datasets

9.2 Raw Data Rule

`data/raw/` is read-only.

Do not manually modify files inside `data/raw/`.

If transformation is needed, output must go to:

```
data/bronze/
data/silver/
```

```
data/gold/
```

9.3 Standard Data Contract

All datasets must eventually be normalized into this schema:

```
text,label
"Congratulations, you won a free prize",1
"Hi team, please review the meeting notes",0
```

Label mapping:

```
0 = ham / not_spam
1 = spam
```

Required columns:

Column	Type	Description
`text`	string	Email content
`label`	int	0 = not_spam, 1 = spam

9.4 Data Quality Rules

Silver and Gold datasets must satisfy:

- Required columns exist: `text`, `label`.
- `text` must not be null or empty.
- `label` must only contain `0` and `1`.
- Duplicate text rows should be handled.
- Train/test split must use stratification.
- Class distribution must be reported.

---

10. Notebook Convention

Notebooks are used for:

- EDA.
- Quick experiments.
- Model comparison.
- Error analysis.

Notebooks are **not** production code.

After an experiment is stable, move the logic into `src/`.

Notebook naming:

```
01_eda.ipynb
02_baseline_logistic_regression.ipynb
03_baseline_naive_bayes.ipynb
04_baseline_knn.ipynb
05_model_comparison.ipynb
```

Rules:

- Keep notebooks clean before committing.
- Remove unnecessary outputs if too large.
- Add short markdown explanations.
- Do not keep secret tokens in notebooks.
- Do not use notebooks as the only source of pipeline logic.

---

11. Model Training Convention

11.1 Main Model

Main model:

```
TF-IDF + Logistic Regression
```

Problem type:

```
Binary classification
```

Labels:

```
0 = not_spam
1 = spam
```

11.2 Model Selection Metrics

Do not select model using accuracy only.

Required metrics:

- Accuracy

- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Primary metric:

```
F1-score
```

Important business note:

```
False positives should be minimized because ham emails are legitimate messages
and should not be incorrectly classified as spam.
```

11.3 MLflow Logging Convention

Each official training run must log:

Params:

```
model_name
tfidf_max_features
tfidf_ngram_range
classifier
random_state
test_size
```

Metrics:

```
accuracy
precision
recall
f1_score
```

Artifacts:

```
classification_report.txt
confusion_matrix.txt
model .pkl
model_metadata.json
```

11.4 Model Artifact Convention

Model files:

```
models/best_spam_classifier.pkl
models/model_metadata.json
```

Do not change these paths without updating:

- `.env.example`
- `app/api.py`
- `src/predict.py`
- README

---

12. API Convention

12.1 FastAPI Endpoints

Required endpoints:

```
GET /health
POST /predict
```

12.2 Request Format

```
{
  "subject": "Congratulations",
  "body": "You won a free iPhone. Click here now."
}
```

12.3 Response Format

```
{
  "label": "spam",
  "spam_probability": 0.94
}
```

Allowed labels:

```
spam
not_spam
```

12.4 Error Handling

Empty input should return clear error.

Example:

```
{
  "detail": "Subject or body must not be empty."
}
```

12.5 Model Loading Rule

The model must be loaded once when FastAPI starts.

Do not load model inside every request.

Bad:

```
def predict_email(data):
    model = joblib.load("models/best_spam_classifier.pkl")
    return model.predict([data.body])
```

Good:

```
model = joblib.load("models/best_spam_classifier.pkl")


def predict_email(data):
    return model.predict([data.body])
```

---

13. Streamlit Convention

Streamlit app must include:

- Project title.
- Short description.
- Subject input.
- Body text area.

- Predict button.
- Spam/not_spam result.
- Spam probability.
- Example spam and not_spam emails.
- API error handling.

Streamlit should call FastAPI using an environment variable:

```
API_URL=http://127.0.0.1:8000/predict
```

Do not hard-code production URLs directly in code.

---

14. Prediction Logging Convention

Prediction logs must be saved to:

```
logs/predictions.csv
```

Required columns:

```
timestamp,subject,body,prediction,spam_probability
```

Rules:

- Create file if it does not exist.
- Append new predictions.
- Do not log secrets or personal sensitive information in real production use.
- For this educational project, logs are for debugging and monitoring demo.

---

15. Environment Variable Convention

Use `.env.example` to document required environment variables.

Example:

```
MODEL_PATH=models/best_spam_classifier.pkl
MLFLOW_TRACKING_URI=http://127.0.0.1:5000
API_HOST=0.0.0.0
API_PORT=8000
STREAMLIT_PORT=8501
```

```
PREDICTION_LOG_PATH=logs/predictions.csv
API_URL=http://127.0.0.1:8000/predict
```

Rules:

- `.env` must not be committed.
- `.env.example` must be committed.
- Any new environment variable must be added to `.env.example`.

---

16. Testing Convention

Use `pytest`.

Test folder:

```
tests/
```

Recommended tests:

```
tests/test_data_pipeline.py
tests/test_predict.py
tests/test_api.py
```

QA must verify:

- Data schema.
- Label mapping.
- Model loading.
- Prediction output.
- API response format.
- UI behavior.
- Prediction logging.

Run all tests:

```
pytest
```

---

17. Documentation Convention

All important project decisions must be documented in `docs/`.

Required docs:

```
docs/fsd.md
docs/tsd.md
docs/architecture.md
docs/coding_and_git_convention.md
docs/test_plan.md
docs/deployment_guide.md
```

Reports:

```
reports/data_report.md
reports/model_report.md
reports/api_ui_report.md
reports/final_test_report.md
```

Rules:

- Use clear headings.
- Use tables when useful.
- Keep documentation updated when scope changes.
- Include commands that can be copied and run.

---

18. Git Convention

18.1 Branch Strategy

Use:

```
main
develop
feature/*
fix/*
docs/*
deploy/*
```

Meaning:

Branch	Purpose
`main`	Stable release/demo code
`develop`	Integration branch
`feature/*`	New feature work
`fix/*`	Bug fixes

Branch	Purpose
`docs/*`	Documentation updates
`deploy/*`	Docker/deployment changes

Flow:

```
feature/* → Pull Request → develop → Pull Request → main
```

No direct push to `main`.

18.2 Branch Naming

Format:

```
<type>/<short-description>
```

Examples:

```
feature/data-medallion-pipeline
feature/baseline-model
feature/mlflow-training
feature/fastapi-serving
feature/streamlit-ui
feature/prediction-logging
fix/api-empty-input
fix/model-path-loading
docs/update-readme
deploy/docker-compose
```

---

19. Commit Convention

Use this format:

```
<type>: <short description>
```

Allowed types:

Type	Meaning
`feat`	New feature

Type	Meaning
`fix`	Bug fix
`docs`	Documentation only
`style`	Formatting only, no logic change
`refactor`	Code improvement without behavior change
`test`	Add or update tests
`chore`	Config, dependencies, project setup
`deploy`	Deployment-related changes

Good examples:

```
feat: add bronze ingestion script
feat: train logistic regression baseline
feat: add mlflow tracking
feat: create fastapi predict endpoint
feat: add streamlit demo ui
fix: handle empty email input
fix: correct model path loading
docs: update project README
test: add api endpoint tests
chore: update requirements file
deploy: add docker compose configuration
```

Bad examples:

```
update
fix bug
done
abc
final
new code
```

---

20. Pull Request Convention

20.1 PR Title

Format:

```
[TYPE] Short description
```

Examples:

```
[FEATURE] Add data medallion pipeline
[FEATURE] Train logistic regression baseline
[FEATURE] Add FastAPI prediction endpoint
[FIX] Handle empty API input
[DOCS] Update README setup guide
[DEPLOY] Add Docker Compose configuration
```

20.2 PR Description Template

Use this template:

```
## Summary
Describe what this PR changes.

## Changes
- Change 1
- Change 2
- Change 3

## How to Test
```bash
python src/train.py
uvicorn app.api:app --reload
```
```

Checklist

- [ ] Code runs locally
- [ ] No unused imports
- [ ] No hard-coded secrets
- [ ] README/docs updated if needed
- [ ] Tests added or updated if needed
- [ ] QA reviewed if output affects data/model/API/UI

```
---

# 21. Code Review Rules

Before merging:

- At least one reviewer must approve.
- Tech Lead reviews important code.
- QA reviews outputs that affect quality.
- No failing tests.
- No secrets in code.
```

```
- No direct push to `main`.

Tech Lead should review:

```text
src/train.py
src/predict.py
app/api.py
app/streamlit_app.py
Dockerfile
docker-compose.yml
```
```

QA should review:

```
data output
model output
API response
UI behavior
prediction logs
final reports
```

---

22. Merge Convention

Standard flow:

```
git checkout develop
git pull origin develop

git checkout -b feature/fastapi-serving

# code...

git add .
git commit -m "feat: add fastapi predict endpoint"
git push origin feature/fastapi-serving
```

Then create Pull Request:

```
feature/fastapi-serving → develop
```

Final release:

```
develop → main
```

---

23. Files Not Allowed in Git

Do not commit:

```
.env
API keys
passwords
AWS credentials
Azure credentials
large raw datasets
large model artifacts
__pycache__/
.venv/
mlruns/
logs/*.log
```

Recommended `.gitignore`:

```
# Python
__pycache__/
*.pyc
*.pyo
*.pyd
.Python

# Virtual environment
.venv/
venv/
env/

# Environment variables
.env

# OS
.DS_Store

# Jupyter
.ipynb_checkpoints/

# Logs
logs/*.log

# MLflow
mlruns/
```

```
# Data outputs
data/raw/
data/bronze/
data/silver/
data/gold/

# Models
models/*.pkl

# Cache
.pytest_cache/
.mypy_cache/
```

Note:

If the team needs a small demo dataset, place it in:

```
data/test_samples/
```

and ensure it is small enough for GitHub.

---

24. Docker Convention

Docker-related files:

```
Dockerfile
docker-compose.yml
```

Expected services:

```
api
streamlit
```

Docker Compose should allow:

```
docker compose up -d --build
```

Required ports:

Service	Port
FastAPI	8000

Service	Port
Streamlit	8501

---

25. Definition of Done

A task is considered Done when:

- Code runs locally.
- No import errors.
- Code follows naming convention.
- No unused imports.
- No hard-coded secrets.
- Output file is created if required.
- Tests are added or updated if needed.
- Documentation is updated if needed.
- Commit message follows convention.
- Pull request is created.
- PR is reviewed and approved.
- QA verifies output if task affects data/model/API/UI.

---

26. Quick Checklist Before Push

Run:

```
black .
isort .
flake8 .
pytest
git status
```

Then commit:

```
git add .
git commit -m "feat: add logistic regression training pipeline"
git push origin feature/mlflow-training
```

---

27. Summary

Use this short rule set when unsure:

```
Code must be readable.
Data must follow the agreed contract.
Model runs must be tracked.
API responses must be consistent.
UI must be simple and testable.
No secrets in Git.
No direct push to main.
Every important change goes through PR review.
QA participates in every sprint.
```